FINSIGHT

Financial Transaction Tracker

Business Requirements Document

Prepared by:

Maryam Ali

Dania Osama Bahjat

Table of Contents

Table of Contents....1
1. Executive Summary....2
2. Business Problem Statement....3
3. Business Goals and Objectives....3
3.1 Goals....4
3.2 Objectives....4
4. Scope....4
4.1 In Scope....5
4.2 Out of Scope....5
5. Stakeholders....5
5.1 Primary Stakeholders....6
5.2 Secondary Stakeholders....6
6. Business Requirements....6
7. Functional Requirements (Business View)....7
8. Non-Functional Requirements....9
9. High-Level Solution Description....10
9.1 Solution Architecture....11
9.2 Key Data Objects (Business View)....11
10. Acceptance Criteria (High-Level)....11
11. Constraints and Assumptions....12
11.1 Constraints....13
11.2 Assumptions....13
12. Risks and Mitigation....13
13. Success Metrics....14

1. Executive Summary

FinSight is a Financial Transaction Tracker designed to help users monitor transactions near real-time, detect and manage potentially fraudulent activity via rule-based risk scoring, and gain actionable spending insights through dashboards. It also supports subscription detection with due-soon notifications and seeds realistic demo transactions for first-time users so the product is immediately explorable without needing real bank integrations.

2. Business Problem Statement

Individuals and teams who want to track spending and detect suspicious activity often face four core challenges:

Problem	Business Impact
1. Lack of fast visibility	Users must manually refresh to see new transactions, leading to delayed awareness of account activity and potential fraud.
2. No explainable fraud indicators	Existing tools either provide no fraud detection or deliver opaque ML-based outputs that users cannot interpret or act upon with confidence.
3. No simple categorisation & filtering	Without structured filtering and categorisation, users cannot efficiently spot spending patterns, anomalies, or budget overruns.
4. No proactive subscription awareness	Recurring monthly payments are invisible until charges appear, causing budget surprises and missed cancellation opportunities.

FinSight addresses all four problems by centralising transaction tracking, applying deterministic rule-based fraud scoring (0–100 with clear risk levels), surfacing fraud alerts for review and resolution, and providing dashboards and subscription notifications.

3. Business Goals and Objectives

3.1 Goals

Goal ID	Goal Description
G-1	Provide a single platform to track transactions and analyse spending patterns with dashboards.
G-2	Detect suspicious transactions automatically using explainable, rule-based scoring and alerts.
G-3	Improve user awareness of recurring financial commitments via subscription detection and due-soon notifications.

3.2 Objectives

Objective ID	Objective Description
O-1	Allow manual transaction entry and ensure every transaction is assessed for fraud risk and logged for compliance.
O-2	Provide comprehensive filtering, sorting, and pagination to quickly find transactions and support polling efficiency.
O-3	Offer a dashboard summary covering income, expenses, balance, fraud metrics, category breakdowns, and spending trends.
O-4	Detect recurring subscriptions (20–40 day patterns) and provide due-soon queries and ignore capability. Allow users to manually trigger subscription detection and ignore unwanted detections.
O-5	Enable near real-time transaction monitoring via a frontend polling toggle (5–10 second interval).

4. Scope

4.1 In Scope

Feature Area	Description
Manual Transaction Creation	Validated transaction entry with automatic fraud scoring, flagging, and alert creation.
Transaction Filtering & Sorting	Filter by type / category / date range / fraudulent flag; sort by date / amount / fraud score / category; combinable filters with pagination.
Near Real-Time Viewing	Frontend polling toggle refreshing transactions every 5–10 seconds.
Rule-Based Fraud Detection	Four-rule additive scoring engine producing a 0–100 risk score mapped to LOW / MEDIUM / HIGH risk levels.
Fraud Alert Lifecycle	Automatic alert creation for high-risk transactions; user review and resolution workflow; alerts permanently preserved and filterable.
Insights Dashboard	Aggregated income, expenses, balance, fraud metrics, and category-level spending breakdowns.
Subscription Detection	Automatic detection of 20–40 day recurring payment patterns grouped by transaction category; ignore capability with confirmation; due-soon queries; UI notifications; manual re-detection trigger; manual subscription creation.
Demo Data Seeding	Automatic generation of 25–50 deterministic demo transactions on first login when no data exists.
Audit Logging	Immutable, UTC-timestamped log of all state-changing user actions for compliance.

4.2 Out of Scope

Item	Reason for Exclusion
Machine learning fraud models	Explicitly deferred to a future phase; rule-based detection is sufficient for Phase 1 and ensures explainability.
Real-time event streaming	True event streaming infrastructure (Kafka, WebSockets) is not required for the polling-based near real-time model.
External payment gateway / bank integrations	Third-party bank data ingestion introduces regulatory and security complexity beyond Phase 1 scope.
Multi-currency support	Single-currency (USD) is sufficient for the Phase 1 user base.
Native mobile applications	Web-responsive design addresses mobile users; native apps require separate platform investment.

5. Stakeholders

5.1 Primary Stakeholders

Stakeholder	Interest / Role
End Users (Consumers)	Track personal spending, add manual transactions, filter transaction history, review and resolve fraud alerts, manage subscriptions, and view dashboard insights. Primary beneficiaries of the product.
Product Owner	Defines and prioritises requirements; accepts deliverables against acceptance criteria; manages scope changes.

5.2 Secondary Stakeholders

Stakeholder	Interest / Role
Developers / Maintainers	Need a backward-compatible, well-tested codebase with a predictable layered architecture and clear API contracts to build and maintain the system effectively.
QA Engineers	Require clearly defined acceptance criteria and testable requirements to design test cases and validate correctness of fraud rules, filters, and subscription detection.
Compliance / Audit	Require immutable audit logs of all state-changing actions for regulatory traceability.

6. Business Requirements

ID	Requirement	Description
BR-1	Immediate Usability	New users must see realistic demo transactions on first login when no data exists, enabling immediate product exploration without real bank integrations.
BR-2	Transaction Capture	Users must be able to add transactions manually and store them reliably with all required metadata (amount, type, category, date).
BR-3	Fraud Risk Visibility	Transactions must be analysed by rule-based fraud detection, scored 0–100, categorised into risk levels (LOW / MEDIUM / HIGH), and flagged when high risk.
BR-4	Fraud Workflow	The system must generate fraud alerts for flagged transactions and support a review and resolution workflow without ever deleting alert history.
BR-5	Insights	The system must provide dashboards summarising the user's financial position and spending patterns by category and over time.
BR-6	Subscription Control	The system must detect recurring subscriptions and notify users about due-soon payments; users must be able to ignore unwanted detections.
BR-7	Near Real-Time Monitoring	Users must be able to see updates automatically via a live-refresh polling mechanism (5–10 second interval) that can be toggled on or off.
BR-8	Compliance Logging	All state-changing user actions must be recorded in an immutable, timestamped audit log for regulatory compliance.

7. Functional Requirements (Business View)

ID	Functional Requirement	Description
FR-1	Demo Data Generation	On login, if the user has 0 transactions, create 25–50 demo transactions spread across the previous 60–90 days. Include varied categories (groceries, utilities, entertainment, salary, rent), both income and expense types, and at least 3 fraud-triggering examples. Generation must be deterministic by user ID.
FR-2	Manual Transaction Creation	User can create a transaction. System validates required fields, computes the fraud score, flags the transaction as fraudulent if score ≥ 70, creates a FraudAlert if score ≥ 40 (MEDIUM or HIGH), and logs the action in the audit trail.
FR-3	Transaction Filtering & Sorting	Filter by type / category / date range / fraudulent flag. Sort by date / amount / fraud score / category. Filters are combinable. Pagination is supported with configurable page size.
FR-4	Near Real-Time Updates	Frontend provides a live refresh toggle. When enabled, the system polls for new transactions every 5–10 seconds and updates the displayed list automatically. Polling stops when the toggle is disabled.
FR-5	Fraud Detection Scoring	System computes a fraud score of 0–100 using four additive rules and maps it to risk levels: LOW (0–39), MEDIUM (40–69), HIGH (70–100).
FR-6	Fraud Alert Creation	For any transaction with fraudScore ≥ 40 (MEDIUM or HIGH risk), create a FraudAlert record with severity matching the computed risk level. Transactions with fraudScore ≥ 70 are additionally marked as fraudulent.
FR-7	Fraud Alert Management	Users can list alerts ordered by most recent first, filter by resolved status or severity, and resolve individual alerts. The system logs each resolution. Alerts are never deleted.
FR-8	Dashboard Summary	Dashboard returns total income, total expenses, current balance (income − expenses), fraud incident count, spending by category, and time-series trend data.
FR-9	Subscription Detection	Identify recurring expense payments by grouping transactions by category and checking for payments 20–40 days apart (with a 20-day deduplication filter to avoid counting multiple payments within the same month). Create a subscription record when 2 or more qualifying transactions in the same category match the pattern. Compute average amount and predicted next due date (last paid + 30 days). Allow users to manually trigger detection. Allow users to ignore subscriptions with a confirmation step. Provide a due-soon query filtering by a configurable number of days.
FR-10	Manual Subscription Creation	User can manually create a subscription record by specifying a name (derived from transaction category), monthly amount, last paid date, and next due date. The system persists the subscription with status ACTIVE, computes the next due

ID	Functional Requirement	Description
		date, and immediately includes it in due-soon queries and the subscriptions list. Manually created subscriptions follow the same ignore workflow as auto-detected ones.

8. Non-Functional Requirements

ID	Non-Functional Requirement
NFR-1 Compatibility	All enhancements must maintain full backward compatibility with existing API endpoints and response contracts. Existing consumers must not be broken by Phase 1 changes.
NFR-2 Performance	Transaction list endpoints must support efficient pagination to minimise payload size. Polling queries must be optimised (indexed lookups) to avoid performance degradation under concurrent live-refresh usage.
NFR-3 Maintainability	The system must follow a layered architecture (Controllers → Services → Repositories) with clean separation of concerns. Business logic must reside exclusively in the service layer.
NFR-4 Testability	All critical paths — fraud detection rules, transaction filters, alert lifecycle, and subscription detection — must have automated test coverage using both example-based unit tests and property-based tests.
NFR-5 Explainability	Fraud scoring must be deterministic and rule-based. Each generated alert must include human-readable reasons explaining which rules were triggered and why.
NFR-6 Auditability	All state-changing actions must produce immutable audit log entries with UTC timestamps. No audit records may ever be modified or deleted.
NFR-7 Availability	Both the backend and frontend services must expose health check endpoints used by Docker container orchestration to confirm liveness before routing traffic.
NFR-8 Deployability	The full system must be deployable with a single docker-compose up --build command, with no manual configuration steps required for the development or demo environment.

9. High-Level Solution Description

9.1 Solution Architecture

FinSight is implemented as a React 18 single-page application frontend communicating with a Spring Boot 3.2.0 REST API backend. Both services follow a layered architecture (Controllers → Services → Repositories) and interact exclusively over HTTP/REST using JSON payloads. Both services are containerised and orchestrated via Docker Compose.

Layer	Technology
Backend Language / Runtime	Java 17
Backend Framework	Spring Boot 3.2.0
ORM / Persistence	Spring Data JPA / Hibernate
Database (Dev / Demo)	H2 In-Memory
Build Tool	Maven 3.8+
Frontend Framework	React 18 (functional components + hooks)
Frontend Web Server	nginx (Alpine)
HTTP Client	Axios 1.x
Containerisation	Docker / Docker Compose

9.2 Key Data Objects (Business View)

Data Object	Business Description
User	Account owner who creates and manages transactions. All financial data, alerts, and subscriptions are scoped to a user.
Transaction	An income or expense financial record containing amount, category, date, optional location, and fraud metadata (score and flag). The primary unit of data in FinSight.
FraudAlert	An immutable record of suspicious transaction activity created automatically when a transaction's fraud score reaches the HIGH threshold. Can be reviewed and resolved by the user but never deleted.
Subscription	A detected recurring monthly payment pattern derived from transaction history. Contains merchant name, average amount, last paid date, predicted next due date, and an ignore status.
AuditLog	An immutable, timestamped record of every state-changing user action (e.g. creating a transaction, resolving an alert, ignoring a subscription) for compliance purposes.

10. Acceptance Criteria (High-Level)

AC ID	Criterion	Pass Condition
AC-1	First-time user demo readiness	Given a user logs in with zero transactions, the system generates 25–50 demo transactions across the previous 60–90 days, with varied categories, both income and expense types, and at least 3 examples that trigger fraud detection.
AC-2	Manual transaction → fraud scoring → alert	Given a user creates a valid manual transaction, the system persists it, computes the fraud score, flags it as fraudulent if score ≥ 70, creates a FraudAlert when score ≥ 40, and records the action in the audit log.
AC-3	Filtering and sorting correctness	Given filter parameters (type / category / date range / fraudulent) and sorting parameters are provided, the returned transactions satisfy all filters simultaneously and are ordered as requested, with correct pagination metadata.
AC-4	Live refresh behaviour	When live refresh is enabled, the UI polls every 5–10 seconds and updates the transaction list when new data is detected. When disabled, polling stops completely.
AC-5	Subscription due-soon notifications	Given subscriptions are detected from 20–40 day recurring patterns grouped by category, the system returns only ACTIVE subscriptions due within a specified number of days, excluding any ignored subscriptions.
AC-6	Fraud alert preservation	Given a fraud alert is created and later resolved, the alert remains retrievable with resolved=true and is never removed from the system.
AC-7	Dashboard accuracy	Given a set of transactions within a date range, the dashboard returns totals for income, expenses, and balance that match the arithmetic sum of the underlying transaction records.
AC-8	Backward compatibility	Given any existing API endpoint is called without new optional parameters introduced in Phase 1, it returns HTTP 200 with a response in the original format.

11. Constraints and Assumptions

11.1 Constraints

Constraint ID	Constraint
CON-1	No external bank transaction ingestion in Phase 1. Demo data seeding and manual entry are the required mechanisms to make the system immediately usable.
CON-2	Fraud detection must be rule-based (not ML-based). All scoring logic must be deterministic, explainable, and fully testable without external model infrastructure.
CON-3	The system must use H2 in-memory database for the development and demo environment. Production database migration is out of scope for Phase 1.
CON-4	JWT authentication is implemented in Phase 1 using Spring Security with BCrypt password hashing and HS256-signed tokens. User context is resolved from the JWT Bearer token on each authenticated request.
CON-5	All enhancements must maintain full backward compatibility with existing API contracts. No breaking changes to existing endpoints are permitted.
CON-6	Deployment must be achievable with Docker Compose on a standard developer machine (2 GB RAM minimum, ports 8080 and 3000 available).

11.2 Assumptions

Assumption ID	Assumption
ASM-1	Users will rely primarily on filtering, sorting, and dashboard insights to analyse their spending patterns rather than raw data export.
ASM-2	A polling-based near real-time UX (5–10 second interval) is acceptable to stakeholders; true sub-second streaming is not required.
ASM-3	The 20–40 day window for subscription detection will successfully identify the vast majority of monthly subscription payments in practice. Category-based grouping provides consistent detection without relying on exact merchant name matching.
ASM-4	Demo data generated deterministically by user ID will provide a sufficiently realistic and varied set of transactions to showcase all system features.
ASM-5	A single-currency (USD) system is sufficient for the Phase 1 target user base.
ASM-6	The H2 in-memory database restart data loss is acceptable for the demo phase; persistent data storage will be addressed in production phases.

12. Risks and Mitigation

Risk ID	Risk	Impact	Mitigation
R-1	False positives from fraud rules	User distrust in fraud alerts; alert fatigue leading to users ignoring genuine high-risk notifications.	Keep scoring fully explainable with per-rule reason descriptions and risk levels. Provide a resolution workflow so users can dismiss false positives without losing history.
R-2	Demo data feels unrealistic	Poor first impression; reduces user confidence in the product during demos and evaluations.	Use category variety with realistic amount ranges per category. Ensure deterministic generation (seeded by userId) so demos are reproducible and consistent.
R-3	Polling causes unnecessary server load	Performance degradation under concurrent live-refresh usage; increased database query volume.	Implement efficient paginated queries with indexed lookups. Allow users to toggle live refresh off. Default polling interval set to 5–10 seconds rather than sub-second.
R-4	Subscription detection false positives	Users see irrelevant subscriptions, eroding trust in the detection feature.	Require 2 transactions with a qualifying 20–40 day interval before creating a subscription record. Apply a 20-day deduplication filter to avoid counting multiple payments within the same billing period. Provide a confirmation modal and ignore capability for incorrect detections.
R-5	Data loss on container restart	Demo or evaluation data is lost, requiring re-seeding; disrupts ongoing evaluations.	Document clearly that H2 is in-memory and data resets on restart. Demo data reseeds automatically on next login. Provide PostgreSQL migration path for persistent environments.
R-6	Backward compatibility breakage	Existing API consumers break when upgrading to the enhanced version.	Introduce all new parameters as optional with defaults. Maintain existing response schemas unchanged. Validate with backward-compatibility property tests (P30).

13. Success Metrics

FinSight Phase 1 is considered successful when all of the following measurable outcomes are achieved:

Metric ID	Success Metric
SM-1 Demo Readiness	100% of new users with zero transactions receive 25–50 demo transactions automatically on first login, with no manual intervention required.
SM-2 Fraud Detection Consistency	Fraud detection produces identical scores and risk levels for identical input patterns across 100% of test iterations (determinism verified by property tests).
SM-3 Alert Integrity	100% of transactions with fraud score ≥ 40 result in a FraudAlert record (MEDIUM or HIGH severity). Transactions with score ≥ 70 are additionally marked fraudulent. No alert is ever deleted from the system.
SM-4 Filter Accuracy	100% of filtered transaction results satisfy all applied filter predicates simultaneously. Zero transactions appear outside the specified date range, type, or category.
SM-5 Subscription Detection	Subscription detection correctly identifies recurring payment patterns with 20–40 day intervals across 100 property-based test iterations.
SM-6 Live Refresh	When live refresh is enabled, the UI reflects new transactions within 10 seconds of creation. Polling stops within 1 polling cycle of toggling off.
SM-7 Test Coverage	All critical business paths (fraud rules, filters, alert lifecycle, subscription detection) have passing automated tests. Target: 10+ tests, 0 failures.
SM-8 Deployment	System deploys successfully with a single docker-compose up --build command, with both services healthy and all features functional within 60 seconds of startup.
SM-9 Backward Compatibility	All existing API endpoints continue to function correctly with their original parameter sets, returning HTTP 200 and original response formats.
SM-10 Dashboard Accuracy	Dashboard totals for income, expenses, and balance match the arithmetic sum of the underlying transaction data with zero rounding discrepancy.